

MASTER PROMPT FOR ANTIGRAVITY

Project: Bank Customer Churn Analytics & Customer Retention Intelligence Platform

Paste this entire document into Antigravity as your project brief. Work through the phases **in order**. Do not start a phase until the previous one is fully working. At the end of each phase, stop, summarize what was built, and wait for approval before continuing.

0. PROJECT CONTEXT (give this to the agent verbatim)

You are building a **Bank Customer Churn Analytics & Customer Retention Intelligence Platform** — a full-stack, data-driven web application, not a slide deck or a static notebook.

Purpose: Analyze bank customer data to explain *why customers leave*, surface *which customers matter most*, and recommend *retention strategies* — not just predict churn.

Target users: Bank managers, relationship teams, marketing, product managers, risk teams, executive leadership. Assume a **non-technical, executive-level audience** will be looking at this UI, so it needs to look like something a Chief Retention Officer would present in a board meeting — not a data-science demo.

Core business questions the product must answer:

- Why are customers leaving?
- Which customers are most valuable?
- Which segments should the bank focus on?
- Which products increase loyalty?
- What retention strategies will reduce churn?

Tech stack (use exactly this unless a phase says otherwise):

Layer	Technology
Data cleaning / feature engineering	Python, Pandas, NumPy
Statistics	SciPy, statsmodels
ML (optional, interpretability-first)	scikit-learn (Logistic Regression, Decision Tree, Random Forest), SHAP for explainability
Database	PostgreSQL
Backend API	Python, FastAPI
Frontend	React + TypeScript + Vite
Styling	Tailwind CSS + a small custom design-token layer (see UI spec below)
Charts	Recharts (primary), D3 only if Recharts genuinely can't do it
Auth (optional, later phase)	simple JWT-based login, role-gated (Executive / Analyst / Manager)
Version control	Git, with a conventional commit per phase

Dataset: use the schema below as the canonical `customers` table. If the user has not supplied a real CSV, generate a realistic **synthetic dataset of 8,000–12,000 customers** with plausible correlations (e.g., low product count + low tenure + high complaints → higher churn probability) so every dashboard has real signal to show, not random noise.

```
customer_id, age, gender, geography, income, credit_score, balance,
num_products, tenure_years, is_active_member, estimated_salary,
card_type, num_complaints, digital_banking_usage, churned
```

1. GLOBAL UI / DESIGN SYSTEM SPEC

(apply this from Phase 4 onward, and treat it as non-negotiable)

Overall tone: Formal, premium, "private banking" feel. Think Bloomberg Terminal meets a modern fintech product — precise, confident, data-dense but never cluttered. Not playful, not startup-pastel, not generic admin-template blue.

Color system

- Base: deep navy / near-black (( #0B1220) → ( #111827) range) for the app shell and sidebar.
- Surface cards: slightly lifted dark panels (( #161F2E)) with a 1px hairline border ((`rgba(255, 255, 255, 0.06)`)) — subtle glass effect, no heavy shadows.
- Primary accent: muted gold / champagne (( #C9A24B)) for key numbers, active nav states, and primary CTAs — used sparingly, like a signature, never flooding the screen.
- Secondary accent: emerald (( #2FBF71)) for positive movement (retention gains, growth) and a restrained crimson (( #D64545)) for churn/risk — colorblind-safe pairing, never pure red/green.
- Neutrals: a 5-step gray scale for text hierarchy (headline / body / muted / disabled).
- Provide a **light theme** variant with the same structure (ivory/white surfaces, navy text, same gold accent) and a theme toggle in the top bar.

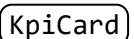
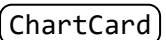
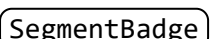
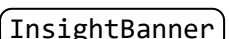
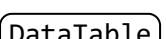
Typography

- Headings: a serif or slab-serif with editorial weight (e.g. "Fraunces", "Playfair Display", or "IBM Plex Serif") — used only for page titles and big KPI numbers, to create the "formal report" feel.
- Body/UI text: a clean grotesk (e.g. "Inter" or "IBM Plex Sans").
- KPI numbers: large, tabular-nums, serif, with a small muted-gray label above/below in the sans font.

Layout

- Persistent left sidebar (collapsible): logo/wordmark, nav grouped by dashboard page (see Phase list), a subtle "last data refresh" timestamp at the bottom.
- Top bar: page title (serif), global date-range filter, geography/segment filter, theme toggle, export-to-PDF button.
- Content area: 12-column responsive grid. KPI cards in a row at the top of every dashboard, charts below in a 2-3 column grid, tables/at-risk lists full width at the bottom.
- Generous whitespace, 8px spacing scale, rounded-lg (not pill-shaped) corners on cards, consistent 24px card padding.

Components to build once, reuse everywhere

-  (big number, label, small trend sparkline or delta arrow)
-  (title, optional subtitle/insight line auto-generated from the data, filter chip, chart body)
-  (Premium / Growth / At-Risk / Dormant, color-coded)
-  (a one-line, plain-English auto-generated insight, e.g. "Customers with 3+ products churn 62% less than single-product customers")
-  with sorting, pagination, CSV export

- `FilterBar` (geography, age band, product count, segment, active/inactive)
- Skeleton loaders for every async component — never a blank white flash.

Motion: subtle only — 150–250ms fade/slide on card mount, animated count-up on KPI numbers, smooth chart transitions on filter change. No bouncy/playful easing.

Explicitly avoid: default shaden purple/blue palette, emoji as UI icons, cartoonish illustrations, gradient-heavy "SaaS landing page" style, Comic-Sans-adjacent fonts, unlabeled charts, more than 2 accent colors on one screen.

Use `lucide-react` for icons only, monochrome, matched to text color.

2. PHASED BUILD PLAN

Execute phases strictly in order. Each phase ends with a working, demo-able increment.

Phase 0 — Project Scaffolding

- Initialize monorepo: `/backend` (FastAPI), `/frontend` (React+Vite+TS+Tailwind), `/data` (raw + processed), `/notebooks` (analysis).
- Set up Postgres schema for `customers`, `complaints`, `transactions` (stub), `segments`.
- Set up Git with `.gitignore`, README describing the project vision (use section 0 above), and environment config (`.env.example`).
- Deliverable: empty app shell running end-to-end (frontend hits a `/health` endpoint on backend, which hits Postgres).

Phase 1 — Data Layer: Cleaning & Ingestion

- Generate/import the dataset described above.
- Build a Python pipeline (`/backend/etl`) that: removes duplicates, handles missing values, standardizes city/geography names, fixes invalid ages, winsorizes outliers in salary/balance, and encodes categoricals.
- Load cleaned data into Postgres.
- Deliverable: a CLI command (`python etl/run.py`) that produces a clean, loaded database from raw data, with a logged summary of rows dropped/fixed.

Phase 2 — Feature Engineering

- Compute and persist these engineered columns per customer:
 - **Customer Lifetime Value (CLV)** — simple formula:
$$\frac{(\text{avg balance} \times \text{tenure} \times \text{product count factor}) - \text{complaint penalty}}{\text{product count factor}}$$
, documented clearly in code comments.
 - **Product Engagement Score** — weighted combo of product count, digital usage, transaction frequency.

- **Financial Health Score** — credit score, balance, income, loan repayment (stub if no loan data).
- **Customer Loyalty Score** — tenure, active membership, product usage, inverse of complaints.
- Deliverable: these four scores available via API and stored in the DB, each normalized 0-100 for easy UI display.

Phase 3 — Statistical Analysis Engine

- Backend service that computes, on demand or cached: correlation matrix (credit score/balance/age vs churn), chi-square tests (churn vs gender/geography/account type), ANOVA (balance across groups), and a logistic regression with interpretable coefficients/odds ratios.
- Expose as `/api/analytics/*` endpoints returning clean JSON (coefficients, p-values, plain-language interpretation strings — this is what powers the `InsightBanner` components later).
- Deliverable: Postman/HTTP-file collection or a `/docs` Swagger page proving each endpoint returns correct, sane values against the seeded data.

Phase 4 — Design System & App Shell

- Implement the full UI spec from Section 1: theme tokens, sidebar, top bar, `KpiCard`, `ChartCard`, `InsightBanner`, `SegmentBadge`, `DataTable`, `FilterBar`, skeleton loaders, light/dark toggle.
- Build a Storybook-style `/design-system` route showing every component in isolation so it can be reviewed before wiring real data in.
- Deliverable: navigable app shell with all 6 pages stubbed (empty state) but the shell, nav, and components look finished.

Phase 5 — Executive Dashboard (page 1)

- KPI row: Total Customers, Churn Rate, Customer Growth (MoM), Average Balance, Total Revenue-at-Risk.
- Revenue-by-segment chart (Premium/Growth/At-Risk/Dormant).
- Top-line `InsightBanner` auto-generated from Phase 3 stats (e.g. biggest churn driver this month).
- Deliverable: fully wired, filterable, real data, matches design spec.

Phase 6 — Customer Insights Dashboard (page 2)

- Age distribution, gender split, geographic churn heatmap/map or bar-by-city, segment distribution.
- Cross-filtering: clicking a bar filters the rest of the page.
- Deliverable: same rigor as Phase 5.

Phase 7 — Product Dashboard (page 3)

- Product ownership breakdown, product-combination retention impact (e.g. table: combo → churn rate), underutilized product flagging.
- Deliverable: same rigor as Phase 5.

Phase 8 — Churn Dashboard (page 4)

- Monthly churn trend line, churn by branch/geography, churn by age band, churn by income band, churn by product.
- Include the logistic-regression-driven "top churn drivers" ranked list with odds ratios shown as horizontal bars.
- Deliverable: same rigor as Phase 5.

Phase 9 — Profitability Dashboard (page 5)

- High-value customer table (sortable by CLV), CLV distribution chart, revenue by segment, a ranked "retention opportunity" list (high CLV + high churn probability = biggest priority, visually flagged).
- Deliverable: same rigor as Phase 5.

Phase 10 — Segmentation & At-Risk Explorer (page 6)

- Run k-means (or similar) clustering into Premium / Growth / At-Risk / Dormant using the engineered scores from Phase 2.
- Interactive scatter (e.g. CLV vs Loyalty Score, colored by segment) with a searchable/filterable customer list beneath, each row showing `SegmentBadge`, key scores, and a "recommended action" text pulled from the Business Recommendations list (see Section 3).
- Deliverable: same rigor as Phase 5.

Phase 11 — SQL / KPI Explorer (power-user page, optional nav item)

- A read-only query console pre-loaded with the canonical business queries (top cities by churn, avg balance by age group, customers with 3+ products, high-income/low-balance customers, churn by card type, most profitable segment) as one-click buttons, results rendered in `DataTable`.
- Deliverable: same rigor as Phase 5, but can be visually simpler/more "console-like" since it's a power-user tool.

Phase 12 — Polish, Predictive Model, Export, Deploy

- Wire in the optional ML model (Random Forest, interpreted via SHAP) purely to power a "churn risk score" badge per customer — reiterate in code comments that **interpretability, not accuracy, is the goal**.

- Add PDF/CSV export on every dashboard, add loading/error states everywhere, run a full responsive pass (tablet breakpoint minimum), and a final accessibility pass (contrast, keyboard nav, aria labels on charts).
 - Deliverable: production build, deploy instructions (Docker Compose for Postgres+backend+frontend), README updated with screenshots.
-

3. BUSINESS RECOMMENDATIONS TO SURFACE IN-PRODUCT

Wire these in as the "recommended action" text wherever the UI shows an at-risk or high-value customer (Phase 10 especially):

- Targeted loyalty offers for high-value customers showing disengagement signals.
 - Cross-sell nudges for single-product customers (multi-product customers churn far less).
 - Prioritized outreach in high-churn regions.
 - Improved first-year onboarding for customers who churn early.
 - Personalized campaigns for segments flagged high-risk by the model.
-

4. WORKING AGREEMENT FOR ANTIGRAVITY

- Confirm the plan back to me before writing code.
- After each phase: show me a summary + how to run/view it, and pause for my go-ahead before moving to the next phase.
- Keep commits scoped to one phase each, with clear messages (feat(phase-5): executive dashboard)).
- If real data isn't provided, clearly label the seeded dataset as synthetic in the README, but make it realistic enough that every chart tells a believable story.
- Never sacrifice the UI spec in Section 1 for speed — this dashboard needs to look boardroom-ready at every phase, not just at the end.